

Fast Exact Dynamic Time Warping on Run-Length Encoded Time Series

Vincent Froese¹ and Brijnesh Jain^{*2}

¹Algorithmics and Computational Complexity, Faculty IV, TU Berlin, Berlin, Germany,
vincent.froese@tu-berlin.de

²Distributed Artificial Intelligence Laboratory, Faculty IV, TU Berlin, Berlin, Germany
brijnesh.jain@dai-labor.de

March 8, 2019

Abstract

Dynamic Time Warping (DTW) is a well-known similarity measure for time series. The standard dynamic programming approach to compute the dtw-distance of two length- n time series, however, requires $O(n^2)$ time, which is often too slow in applications. Therefore, many heuristics have been proposed to speed up the dtw computation. These are often based on approximating or bounding the true dtw-distance or considering special inputs (e.g. binary or piecewise constant time series). In this paper, we present a fast and exact algorithm to compute the dtw-distance of two run-length encoded time series. This might be used for fast and accurate indexing and classification of time series in combination with preprocessing techniques such as piecewise aggregate approximation (PAA).

Keywords: Dynamic Programming, Time Series Similarity, Efficient Algorithms

1 Introduction

Time series data is ubiquitous appearing in essentially all scientific domains. Comparing time series requires a measure to determine the proximity of two time series. Dynamic Time Warping (DTW) [8] is an established method which is used in numerous time series mining applications [1, 2, 3].

The computational complexity is considered to be the major drawback of DTW on extremely long time series even in optimized nearest neighbor search applications that apply sophisticated pruning and lower-bounding techniques [10]. Long Time series of length $\gg 10,000$ occur, for example, when measuring electrical power of household appliances with a sampling rate of a few seconds collected over several months, twitter activity data sampled in milliseconds, and human activities inferred from a smart home environment [7]. All these time series have in common that they contain long constant segments.

Recently, two specialized algorithms have been devised to cope with long time series that contain constant segments [7, 9]. Both algorithms operate on run-length encoded time series. The run-length encoding represents a run of identical values (constant segment) by storing only a single value together with the length of the run. The basic idea of both algorithms is to exploit the repetitions of values within a time series to speed up computation of the DTW distance. In

^{*}Supported by the DFG project JA 2109/4-1.

experiments, both algorithms were magnitudes faster than the standard dynamic program for computing the DTW distance [7, 9].

The first algorithm is AWarp [7]. This algorithm is exact for binary-valued time series with repetitions of zero. The second algorithm is Blocked DTW (BDTW) [9]. This algorithm generalizes AWarp in that it is exact for two-valued time series with repetitions of either of both (possibly non-zero) values. A formal proof of exactness, however, is missing for both algorithms. A practical limitation of both algorithms is that they are only exact on the special class of binary and two-valued time series, resp., and otherwise return approximate solutions for the general case of multi-valued time series with repetitions of any value.

Our Contributions. We develop an improved dynamic program that computes the exact DTW distance between two time series x and y of length m and n in $O(\ell m + k n)$ time (and linear space $O(m + n)$), where k and ℓ are the coding lengths of x and y . Note that this yields a linear-time algorithm for time series of constant coding length $k, \ell \in O(1)$. For $k \leq cm$ and $\ell \leq cn$, $c < 1$, the running time might still be faster by a constant factor in practice (in the worst case, the running time equals the standard time of $O(mn)$).

2 Preliminaries

We give some preliminary definitions and introduce notation.

Notation. An $m \times n$ table T consists of m rows and n columns, where $t_{i\bullet}$ denotes the i -th row and $t_{\bullet j}$ denotes the j -th column. We denote the j -th entry of the i -th row by $T[i, j]$.

Time Series. A time series is an ordered finite sequence $x = (x_1, \dots, x_n)$ of rationals. The *run-length encoding* of a time series x is an ordered sequence $\tilde{x} = ((\tilde{x}_1, n_1), \dots, (\tilde{x}_k, n_k))$ of pairs $(\tilde{x}_i, n_i)$ with $x_1 = \tilde{x}_1 \neq \tilde{x}_2 \neq \dots \neq \tilde{x}_k$, where n_i is a positive integer denoting the number of consecutive repetitions of the value $\tilde{x}_i$ in x . Note that $\sum_{i=1}^k n_i = n$. We call $|x| = n$ the *length* of x and we call $|\tilde{x}| = k$ the *coding length* of x .

Dynamic Time Warping. The dynamic time warping distance [8] is a distance measure between time series using non-linear alignments. It can be computed via dynamic programming in $O(mn)$ time for two time series of length m and n based on an $m \times n$ table. Dynamic time warping is defined via the concept of a warping path.

Definition 1. A warping path of order $m \times n$ is a sequence $p = (p_1, \dots, p_L)$, $L \in \mathbb{N}$, of index pairs $p_\ell = (i_\ell, j_\ell) \in \{1, \dots, m\} \times \{1, \dots, n\}$, $1 \leq \ell \leq L$, such that

- (i) $p_1 = (1, 1)$,
- (ii) $p_L = (m, n)$, and
- (iii) $(i_{\ell+1} - i_\ell, j_{\ell+1} - j_\ell) \in \{(1, 0), (0, 1), (1, 1)\}$ for each $1 \leq \ell \leq L - 1$.

The set of all warping paths of order $m \times n$ is denoted by $\mathcal{P}_{m,n}$. A warping path $p \in \mathcal{P}_{m,n}$ defines an *alignment* between two time series $x = (x_1, \dots, x_m)$ and $y = (y_1, \dots, y_n)$ in the following way: Every pair $(i, j) \in p$ aligns element x_i with y_j . The *dtw-distance* between x and y is defined as

$$\text{dtw}(x, y) := \min_{p \in \mathcal{P}_{m,n}} \sqrt{\sum_{(i,j) \in p} (x_i - y_j)^2}.$$

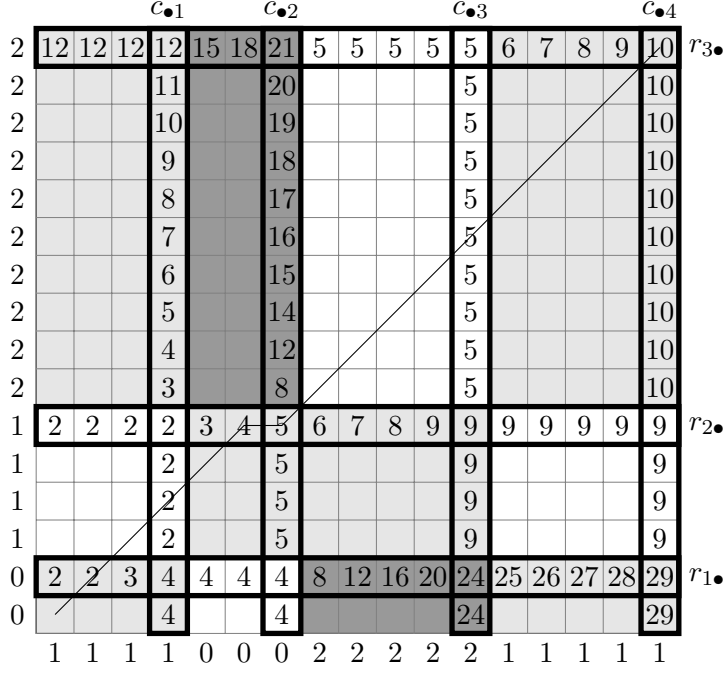


Figure 1: Example of a local cost matrix for two time series x and y with run-length encodings $\tilde{x} = ((1, 4), (0, 3), (2, 5), (1, 5))$ and $\tilde{y} = ((0, 2), (1, 4), (2, 10))$. Colors indicate the local costs $(x_i - y_j)^2$ (white = 0, light gray = 1, dark gray = 4). The numbers are costs of optimal warping paths to the corresponding entry. An optimal 17×16 warping path is shown. Note that it is sufficient to compute the thick-lined columns (C) and rows (R) in order to determine $\text{dtw}(x, y)$.

3 Fast DTW on Run-length Encoded Time Series

In the following, we prove our main result.

Theorem 3.1. *Let $x = (x_1, \dots, x_m)$ be a time series with coding length k and let $y = (y_1, \dots, y_n)$ be a time series of coding length ℓ . The dynamic time warping distance between x and y can be computed from $\tilde{x}$ and $\tilde{y}$ in $O(kn + \ell m)$ time.*

Proof. Let $\tilde{x} = ((\tilde{x}_1, m_1), \dots, (\tilde{x}_k, m_k))$ and $\tilde{y} = ((\tilde{y}_1, n_1), \dots, (\tilde{y}_\ell, n_\ell))$ be the run-length encodings of x and y . We define $a_i := \sum_{j=1}^i m_j$ for $1 \leq i \leq k$ and $b_i := \sum_{j=1}^i n_j$ for $1 \leq i \leq \ell$. First, recall that the classic dynamic program to compute the dtw-distance between x and y builds up an $n \times m$ table D , where $D[i, j] = \text{dtw}((x_1, \dots, x_j), (y_1, \dots, y_i))^2$.

We compute $\text{dtw}(x, y)$ via dynamic programming using two tables: An $\ell \times m$ table R and an $n \times k$ table C , where

$$R[i, j] = \text{dtw}((x_1, \dots, x_j), (y_1, \dots, y_{b_i}))^2, \text{ and} \\ C[i, j] = \text{dtw}((x_1, \dots, x_{a_j}), (y_1, \dots, y_i))^2.$$

That is, the ℓ rows of R are the rows of D with index $b_1, \dots, b_\ell$ and the k columns of C are the columns of D with index $a_1, \dots, a_k$ (see Figure 1). Clearly, $\text{dtw}(x, y)$ can then be obtained from $R[\ell, m]$ ($= C[n, k]$).

In the following, we only show how to fill the table R (since filling C is completely symmetric). The following observation shows a monotonicity within rows of R and columns of C which is the essential argument for the correctness.

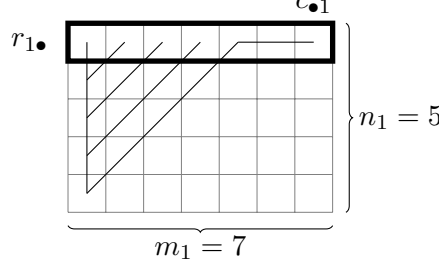


Figure 2: Example of filling the first m_1 columns of row $r_{1\bullet}$ (framed in thick lines). For each entry of $r_{1\bullet}$, the relevant warping paths are drawn. Note that every step costs $\delta = (\tilde{x}_1 - \tilde{y}_1)^2$.

Observation 3.2. Let $a_0 = b_0 := 0$. Then, for every $i \in [\ell]$ and $l \in [k]$, it holds:

- (i) For each j with $a_{l-1} + 1 < j \leq a_l$, it holds $R[i, j-1] \leq R[i, j]$.
- (ii) For each j with $b_{i-1} + 1 < j \leq b_i$, it holds $C[j-1, l] \leq C[j, l]$.

Proof. We only prove (i) (the argument for (ii) is analogous).

Let $p = (p_1 = (1, 1), \dots, p_L = (j, b_i))$ be an optimal warping path with cost $R[i, j]$. Note that p has to contain a pair $p_l = (j_l, i_l)$ such that $j_l = j-1$. Let l be the largest index of such a pair. If $p_l = (j-1, b_i)$, then this implies $R[i, j-1] \leq R[i, j]$. Otherwise, we can assume that $p_{l+1} = (j, i_l + 1), p_{l+2} = (j, i_l + 2), \dots, p_L = (j, b_i)$.

Now, consider the warping path $p' = (p_1, \dots, p_l, (j-1, i_l + 1), (j-1, i_l + 2), \dots, (j-1, b_i))$ and note that it also has cost $R[i, j]$ since $x_{j-1} = x_j$ (since $a_{l-1} + 1 < j \leq a_l$). $\square$

Initialization. We initialize as follows:

$$R[1, j] = (\tilde{x}_1 - \tilde{y}_1)^2 \cdot \max(n_1, j), \quad 1 \leq j \leq m_1.$$

It is easy to see that this is correct since $\text{dtw}((x_1, \dots, x_j), (y_1, \dots, y_{n_1}))^2$ is determined by a shortest (that is, length $\max(n_1, j)$) warping path, where each step costs $(x_1 - y_1)^2$ (see Figure 2).

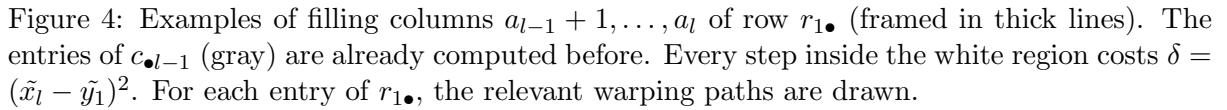
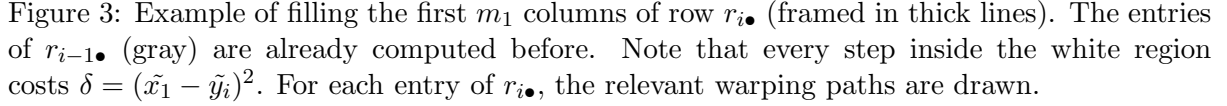
First m_1 columns. We continue to fill the first m_1 columns of R by iterating over the rows $i = 2, \dots, \ell$. Let $M = \min(m_1, n_i + 1)$ and $\delta := (\tilde{x}_1 - \tilde{y}_i)^2$. We set

$$R[i, j] = R[i-1, 1] + n_i \cdot \delta \quad 1 \leq j \leq M, \quad (1)$$

$$R[i, j] = \min(R[i, j-1] + \delta, R[i-1, j-n_i] + n_i \cdot \delta) \quad M < j \leq m_1. \quad (2)$$

To see that (1) and (2) are correct, note that any optimal warping path for $R[i, j]$ has to go over some $R[i-1, j']$, $1 \leq j' \leq j$, and takes at least n_i steps with cost δ from there (see Figure 3). By Observation 3.2, for all $1 \leq j \leq n_i + 1$, it is therefore optimal to go over $R[i-1, 1]$ with exactly n_i additional steps. Hence, (1) is correct. For $j > n_i + 1$, an optimal warping path goes over some $R[i-1, j']$, $1 \leq j' \leq j - n_i$ (note that Observation 3.2 implies that any $j' > j - n_i$ cannot be better). Note that for $j' < j - n_i$, an optimal warping path over $R[i-1, j']$ can always go over $R[i, j-1]$. Hence, (2) is correct.

First row. We continue to fill the remaining columns of the first row of R . (Note that at this point we already have computed the first n_1 rows of C .) For each $l = 2, \dots, k$, we fill the columns $a_{l-1} + 1, \dots, a_l$ as follows: Let $\delta := (\tilde{x}_l - \tilde{y}_1)^2$.


$$R[1, a_{l-1} + j] = \min(R[1, a_{l-1} + j + 1], C[n_1 - j, l - 1] + j \cdot \delta) \quad n_1 - 2 \geq j \geq 1. \quad (4)$$
$$R[1, a_l] = \min_{1 \leq j \leq n_1 - m_l} (C[j, l - 1] + (n_1 - j) \cdot \delta) \quad (5)$$

$$R[1, a_{l-1} + j] = \min(R[1, a_{l-1} + j + 1], C[n_1 - j, l - 1] + j \cdot \delta) \quad m_l - 1 \geq j \geq 1. \quad (6)$$

Remaining entries. To fill the remaining entries of R , we iterate over each row $i = 2, \dots, \ell$ and each $l = 2, \dots, k$ (note that the first column of C is already computed). Let $\delta := (\tilde{x}_l - \tilde{y}_i)^2$ and let $\mu := \min(R[i-1, a_{l-1}], R[i-1, a_{l-1}+1]) + n_i \delta$.

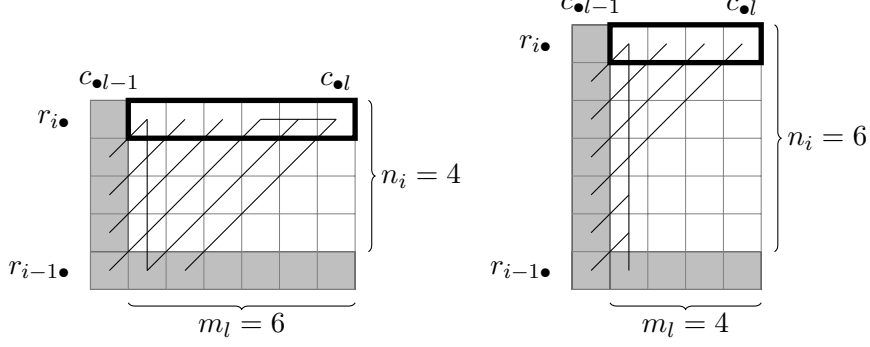


Figure 5: Examples of filling columns $a_{l-1} + 1, \dots, a_l$ of row $r_{i\bullet}$ (framed in thick lines). The entries of $c_{\bullet l-1}$ and $r_{i-1\bullet}$ (gray) are already computed before. Every step inside the white region costs $\delta = (\tilde{x}_l - \tilde{y}_i)^2$. For each entry of $r_{i\bullet}$, the relevant warping paths are drawn.

If $m_l \geq n_i$, then we set

$$R[i, a_{l-1} + j] = \min(C[b_i - j, l - 1] + j \cdot \delta, R[i, a_{l-1} + j + 1]) \quad 1 \leq j < n_i \quad (7)$$

$$R[i, a_{l-1} + n_i] = \min(C[b_{i-1} + 1, l - 1] + n_i \cdot \delta, \mu) \quad (8)$$

$$R[i, a_{l-1} + j] = \min(R[i, a_{l-1} + j - 1] + \delta, R[i - 1, a_{l-1} + j - n_i] + n_i \cdot \delta) \quad n_i < j \leq m_l \quad (9)$$

If $m_l < n_i$, then let $\lambda := \min_{1 \leq j \leq n_i - m_l} (C[b_{i-1} + j, l - 1] + (n_i - j) \cdot \delta)$ and set

$$R[i, a_l] = \min(\lambda, \mu) \quad (10)$$

$$R[i, a_{l-1} + j] = \min(C[b_i - j, l - 1] + j \cdot \delta, R[i, a_{l-1} + j + 1]) \quad m_l - 1 \geq j \geq 1 \quad (11)$$

The correctness follows from similar arguments as for (3)–(6). Note that, by Observation 3.2, an optimal warping path to $R[i, a_{l-1} + j]$ can be assumed to either go over $C[j', l - 1]$ for some $b_{i-1} < j' \leq \max(b_{i-1} + 1, b_i - j)$, or over $R[i - 1, j']$ for some $a_{l-1} \leq j' \leq \max(a_{l-1} + 1, a_{l-1} + j - n_i)$ (see Figure 5). Hence, (7), (8), (10), and (11) are correct. For (9), note that an optimal warping path to $R[i, a_{l-1} + j]$ can be assumed to either go over $R[i, a_{l-1} + j - 1]$ or $R[i - 1, a_{l-1} + j - n_i]$. This completes the correctness (table C can be filled analogously by column-wise iteration).

Concerning the running time, filling tables R and C requires to compute $kn + \ell m$ many entries. Note that an entry can be computed in constant time, except for the case (5) and (10). Observe, however, that computing columns $a_{l-1} + 1, \dots, a_l$ of row $r_{i\bullet}$ requires to look up each entry $b_{i-1}, \dots, b_i$ of column $c_{\bullet l-1}$ only once. Thus, every computed entry is only looked up a constant number of times. Overall, the running time is in $O(kn + \ell m)$. Note also that at any point we only need to store the two current rows of R (and two current columns of C). Thus, we need only $O(m + n)$ space. $\square$

4 Conclusion

We presented a fast dynamic program to compute the exact dtw-distance between two given run-length encoded time series. Our method might yield improved performance in practice, especially in conjunction with dimension reduction such as piecewise aggregate approximation [4, 5, 6, 11]. In future work, we will evaluate our algorithm empirically in experiments and compare it to other methods, e.g. AWarp [7] and BDTW [9].

References

- [1] A. Abanda, U. Mori, and J. A. Lozano. A review on distance based time series classification. *Data Mining and Knowledge Discovery*, pages 1–35, 2018.
- [2] S. Aghabozorgi, A. S. Shirkhorshidi, and T. Y. Wah. Time-series clustering—a decade review. *Information Systems*, 53:16–38, 2015.
- [3] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh. The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Mining and Knowledge Discovery*, 31(3):606–660, 2017.
- [4] K. Chakrabarti, E. Keogh, S. Mehrotra, and M. Pazzani. Locally adaptive dimensionality reduction for indexing large time series databases. *ACM Transactions on Database Systems*, 27(2):188–228, 2002.
- [5] C. Faloutsos, H. Jagadish, A. Mendelzon, and T. Milo. A signature technique for similarity-based queries. In *Proceedings of the Compression and Complexity of Sequences 1997 (SEQUENCES '97)*, pages 11–13. IEEE, 1997.
- [6] E. Keogh, K. Chakrabarti, M. Pazzani, and S. Mehrotra. Dimensionality reduction for fast similarity search in large time series databases. *Knowledge and Information Systems*, 3(3):263–286, 2001.
- [7] A. Mueen, N. Chavoshi, N. Abu-El-Rub, H. Hamooni, and A. Minnich. AWarp: Fast warping distance for sparse time series. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*, pages 350–359, 2016.
- [8] H. Sakoe and S. Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.
- [9] A. Sharabiani, H. Darabi, S. Harford, E. Douzali, F. Karim, H. Johnson, and S. Chen. Asymptotic dynamic time warping calculation with utilizing value repetition. *Knowledge and Information Systems*, 57(2):359–388, 2018.
- [10] D. F. Silva, R. Giusti, E. Keogh, and G. Batista. Speeding up similarity search under dynamic time warping by pruning unpromising alignments. *Data Mining and Knowledge Discovery*, 32(4):988–1016, 2018.
- [11] B.-K. Yi and C. Faloutsos. Fast time sequence indexing for arbitrary $\mathcal{L}_p$ norms. In *Proceedings of the 26th VLDB Conference*, pages 385–394, 2000.